Lekcja 13. Statyczne elementy członkowskie klas - zadania

Programowanie aplikacji - klasa 4 technik programista - INF.04

Czas pracy: 45-90 minut. Zadania wykonuj w C++ w dowolnym środowisku IDE lub kompilatorze online obsługującym klasy.

A. Pytania kontrolne

1. Co oznacza, że element członkowski klasy jest statyczny?

2. Czym różni się zmienna instancyjna od statycznej zmiennej członkowskiej?

3. Dlaczego statyczną zmienną członkowską zwykle inicjalizuje się poza definicją klasy?

4. Do czego służy operator zakresu :: w zapisie Pracownik::s_szkola?

5. Dlaczego odwołanie do pola statycznego przez nazwę klasy jest czytelniejsze niż przez obiekt?

6. Czy zmiana wartości zmiennej statycznej dotyczy jednego obiektu czy całej klasy? Wyjaśnij.

7. Czym jest statyczna funkcja członkowska?

8. Dlaczego metoda statyczna nie może bezpośrednio korzystać ze zwykłego pola instancyjnego?

9. Jaką rolę pełnią metody dostępowe, takie jak getter i setter?

10. Podaj przykład sytuacji, w której użycie pola statycznego w klasie ma sens.

B. Analiza kodu

Zadanie B1. Wskaż, które pola są statyczne, a które instancyjne. Następnie napisz, ile kopii każdego pola istnieje po utworzeniu dwóch obiektów klasy Uczen.

```
class Uczen {
public:
    static string s_klasa;
    string imie;
    string nazwisko;
    int numer;
};
```

Zadanie B2. Przeanalizuj kod i określ, co zostanie wypisane na ekranie. Wyjaśnij, dlaczego.

```
class Grupa {
public:
    static string nazwa;
};

string Grupa::nazwa = "4TP";

int main() {
    Grupa g1;
    Grupa g2;
    cout << Grupa::nazwa << endl;
    Grupa::nazwa = "4TA";
    cout << g1.nazwa << endl;
    cout << g2.nazwa << endl;
}
```

Zadanie B3. Znajdź błąd w poniższym kodzie i zaproponuj poprawkę.

```
class Licznik {
    int wartosc;
```

```
public:
    static void pokaz() {
        cout << wartosc << endl;
    }
};
```

Zadanie B4. Wyjaśnij, dlaczego poniższy zapis jest poprawny, ale mniej zalecany stylistycznie.

```
Pracownik p;
p.s_stanowisko = "technik";
```

C. Zadania programistyczne

Zadanie 1 - klasa Uczen z polami statycznymi

Zdefiniuj klasę Uczen zawierającą pola instancyjne: imie, nazwisko, numer_w_dzienniku oraz pola statyczne: s_klasa i s_zawod. Utwórz dwa obiekty klasy Uczen, przypisz im różne dane osobowe, a następnie wyświetl dane obu uczniów wraz z informacją o klasie i zawodzie.

Wymagania: pola s_klasa i s_zawod mogą być publiczne; odwołuj się do nich przez nazwę klasy; pokaż w kodzie, że zmiana wartości statycznej dotyczy obu obiektów.

Zadanie 2 - prywatna zmienna statyczna i akcesory

Zmodyfikuj rozwiązanie z zadania 1 tak, aby s_klasa była prywatną statyczną zmienną członkowską. Dodaj dwie publiczne statyczne metody: ustawKlase(string nowaKlasa) oraz zwrocKlase().

W programie głównym ustaw klasę za pomocą metody statycznej, a następnie odczytaj ją przez metodę zwracającą wartość.

Zadanie 3 - licznik obiektów

Zaprojektuj klasę KontoUzytkownika. Każdy obiekt ma przechowywać login użytkownika. Klasa ma mieć prywatną statyczną zmienną liczbaKont, która zlicza, ile obiektów klasy utworzono. Dodaj statyczną metodę zwrocLiczbeKont().

Wskazówka: zwiększaj licznik w konstruktorze klasy. Jeżeli konstruktory nie były jeszcze szczegółowo omawiane, możesz zastosować prostą metodę inicjalizuj(), która po nadaniu loginu zwiększy licznik.

Zadanie 4 - metoda statyczna bez obiektu

Zdefiniuj klasę Kalkulator z dwiema statycznymi metodami: dodaj(int a, int b) oraz pomnoz(int a, int b). Wywołaj je bez tworzenia obiektu klasy.

```
cout << Kalkulator::dodaj(4, 7) << endl;
cout << Kalkulator::pomnoz(3, 5) << endl;
```

Zadanie 5 - refaktoryzacja

Masz klasę, w której nazwa szkoły została zapisana jako zwykłe pole w każdym obiekcie. Przepisz klasę tak, aby nazwa szkoły była wspólnym polem statycznym.

```
class Uczen {
public:
    string imie;
    string nazwisko;
    string szkola;
};
```

Zadanie 6 - mini-projekt

Napisz krótki program do ewidencji członków koła informatycznego. Klasa CzlonekKola ma przechowywać imię, nazwisko i funkcję ucznia w kole. Wspólne dla wszystkich obiektów mają być: nazwa koła oraz opiekun koła. Zastosuj prywatne statyczne pola i publiczne

statyczne metody dostępowe.

D. Checklist przed oddaniem

- Czy odróżniasz pola instancyjne od statycznych?
- Czy statyczne pola zostały poprawnie zainicjalizowane?
- Czy do elementów statycznych odwołujesz się przez nazwę klasy?
- Czy metoda statyczna nie korzysta bezpośrednio ze zwykłych pól obiektu?
- Czy prywatne pola statyczne mają metody dostępowe?
- Czy program kompiluje się i pokazuje, że wartość statyczna jest wspólna dla obiektów?